

ARQUITECTURA DEL COMPUTADOR

INFORME PRÁCTICA DE LABORATORIO 3

Nombre: Victor Pinto, C.I: 31709216

Pregunta 1

Explique cómo se organiza la memoria cuando un sistema utiliza *memory-mapped I/O*. ¿En qué región de memoria se suelen mapear los dispositivos? ¿Qué implicaciones tiene para las instrucciones `lw` y `sw`?

En un sistema *memory-mapped*, tanto la memoria RAM como los dispositivos de entrada y salida comparten el mismo espacio de direcciones, en vez de tenerlas completamente separadas. Esto implica que las instrucciones `lw` y `sw` pueden comunicarse directamente con los periféricos e interactuar con ellos, leyendo o escribiendo. A raíz de esto, una dirección física puede contener un dato de la RAM o un dato de los dispositivos de I/O. En el caso de MIPS, los dispositivos de entrada y salida se mapean en direcciones altas; sin embargo, esto solo sucede en el simulador MARS.

Pregunta 2

¿Cuál es la principal diferencia entre *memory-mapped I/O* y la entrada/salida por puertos? ¿Qué ventajas y desventajas tiene cada enfoque? ¿Por qué MIPS32 utiliza principalmente *memory-mapped I/O*?

La diferencia radica en cómo se organizan los espacios de direcciones. En la implementación *Port-mapped*, el espacio de direcciones se encuentra en una región individual y aislada de la memoria RAM de la CPU. La ventaja de la *memory-mapped* recae en que no se deben crear instrucciones dedicadas para la I/O, lo que reduce la cantidad de dichas instrucciones (filosofía RISC, fundamental para MIPS). Todas las operaciones naturales que la CPU ejerce también aplican para la *memory-mapped*, lo cual hace al lenguaje más universal. Una de las desventajas de esta implementación es que la reserva de la dirección de instrucciones para los dispositivos en memoria ocupa un espacio extra que podría ser ocupado por otra información. El *port-mapped* resulta ventajoso para mantener una lógica separada entre lo que la CPU controla y las operaciones para la I/O.

Pregunta 3

En un sistema con *memory-mapped I/O*: ¿Qué problemas pueden surgir si dos dispositivos usan direcciones solapadas? ¿Cómo se evita este conflicto?

Un solapamiento ocurre cuando dos dispositivos comparten una dirección física de memoria. Esto puede producir problemas como comportamientos impredecibles o no deseados al intentar modificar uno de los dispositivos, repercutiendo en otro u otros cuyas

direcciones se encuentran solapadas. En función de la arquitectura, este conflicto puede solucionarse asignándole a cada dispositivo un rango único, o también mediante un decodificador de direcciones que gestione adecuadamente el acceso.

Pregunta 4

¿Por qué se considera que el *memory-mapped I/O* simplifica el diseño del conjunto de instrucciones de un procesador? ¿Qué tipo de instrucciones adicionales serían necesarias si se usara E/S por puertos?

El diseño *port-mapped* requiere instrucciones dedicadas y específicas para trabajar con su conjunto de direcciones. Esto no ocurre con la implementación *memory-mapped*, pues los dispositivos y sus registros se alojan en el mismo rango de memoria que la CPU puede leer y procesar. Por ejemplo, las instrucciones `In` y `Out` leen o escriben un dato del puerto respectivamente; estas instrucciones no existen en MIPS al ser una arquitectura de *memory-mapped*.

Pregunta 5

¿Qué ocurre a nivel del bus de datos y direcciones cuando el procesador accede a una dirección de memoria que corresponde a un dispositivo? ¿Cómo sabe el hardware que debe acceder a un periférico en lugar de la RAM?

El CPU, por sí solo, ignora realmente el origen del dato que está procesando o escribiendo. Esto significa que puede recibir o enviar, por medio de los buses de datos, cualquier información por parte de un periférico o de la RAM sin distinguirlos. Es un decodificador de direcciones el encargado de activar la región de donde proviene o se desea escribir un dato, a través de un *chip select* que enciende a 1 el origen o destino del dato.

Pregunta 6

¿Es posible que un programa normal (sin privilegios) acceda a un dispositivo mapeado en memoria? ¿Qué mecanismos de protección existen para evitar accesos no autorizados?

Esto puede ocurrir cuando no existe una protección que cubra las direcciones de los dispositivos y cuando la línea entre el modo Kernel y el del usuario es inexistente o difusa. Para proteger los dispositivos, cada proceso cuenta con una tabla de páginas que contiene, entre otras cosas, si el proceso tiene permiso para ciertas direcciones, en este caso, para la dirección de un dispositivo. El sistema se encarga de bloquear esas peticiones.

Pregunta 7

¿Qué técnicas se pueden emplear para evitar esperas activas innecesarias al interactuar con dispositivos?

Es posible introducir técnicas como la concurrencia, para que el CPU siga trabajando en segundo plano, o en otro hilo, mientras la interrupción del dispositivo ocurre. Por otra parte, la interrupción puede quedar delegada al dispositivo, siendo este el que deba avisar al CPU para retomar el control, mientras que la propia CPU realiza otras tareas.

Implementar una DMA (*Direct Memory Access*) puede evitar las esperas, permitiendo las transferencias de datos entre periféricos y RAM sin que la CPU esté involucrada.

Pregunta 8

Análisis y Discusión de los Resultados.

El desarrollo de estas prácticas en MIPS32 nos permitió interactuar con la entrada y salida estándar. Un dato escrito desde un dispositivo, como el teclado, fue exitosamente cargado a una dirección de memoria designada por el `.data`, sin necesidad de utilizar instrucciones más complejas o dedicadas, sino las instrucciones de manejo de memoria con las que hemos trabajado anteriormente: `sw` y `lw`, que funcionan de manera intuitiva con la I/O producto del diseño *Memory-mapped*. Nótese cómo una interrupción, en este caso producida por el teclado para leer un dato, paraliza el flujo del programa indefinidamente, lo que evidencia la capacidad del dispositivo para interactuar con el CPU.